Self-Taught Semi-Self Speculative Decoding (S4D): A Learned Routing Policy for Hierarchical Verification

viasd-rl experiments

June 22, 2026

Abstract

We present S^{4}D (Self-Taught Semi-Self Speculative Decoding), the first reinforcement-learned routing policy for hierarchical speculative decoding. We make three contributions. First, we introduce a learned per-token gating policy that recasts the choice among verification tiers as a sequential decision problem, so that a small multilayer perceptron, which reads only features that do not require the target model, replaces the hand-tuned confidence thresholds of prior hierarchical speculative decoding and never invokes the target model in order to make its decision. Second, we train this policy by reinforcement learning with a reward that balances fidelity to the target model against the latency of expensive verifier calls, and we study how that reward should be assigned to individual decisions. Third, we derive the intermediate verifier from the target model itself through an offline layer-skip search, so that the target verifies its own draft through a reduced copy of itself. Prior hierarchical speculative decoding selects among its verification tiers using hand-tuned confidence thresholds. We show that this fixed rule is markedly suboptimal and that the learned gate Pareto-dominates it, matching the lossless accuracy of standard speculative decoding (0.91) at four times fewer full-model calls and reaching up to a 3.5-fold speedup. To our knowledge, this is the strongest accuracy-efficiency trade-off reported for hierarchical speculative decoding.

1 Introduction

Autoregressive inference in large language models is memory-bandwidth bound. Generating each token requires streaming all of the model's weights out of high-bandwidth memory, so latency is dominated by a long and strictly sequential chain of large-model forward passes. For a 14-billion-parameter model, this sequential cost is the principal bottleneck behind both interactive latency and serving expense. Speculative decoding addresses it by introducing a cheap drafter that proposes several tokens at once, which the large verifier then checks in a single parallel forward pass. Because one expensive pass is amortized over several accepted tokens, the procedure is lossless and yields a wall-clock speedup of roughly 1.3 to 2 times in practice. Its only lever is the acceptance rate: whenever a proposed token is rejected, the method must fall back to a full-model step, so reducing rejections is what buys speed.

VIA-SD improves on this scheme by inserting a slim intermediate verifier, denoted q', which is a layer-skipped copy of the target model q. Many of the tokens that the target would reject are in fact close calls, for which the drafter is plausible but not fully reliable, and VIA-SD resolves these "middle-zone" tokens cheaply with q' rather than escalating immediately to the full model. The result is a three-tier hierarchy in which each token is either accepted from the drafter, regenerated with q', or escalated to q, and the number of full-model invocations falls accordingly. The decision among these three tiers, however, is made by two fixed confidence thresholds, α_1 and α_2, applied

to q'. This rule is global and context-blind, and it must be re-tuned for every new model and task. Selecting a verification tier for each token from its local context is precisely a sequential, context-dependent control problem, which a learned policy should handle better than a pair of fixed constants.

We therefore replace the fixed thresholds with a learned per-token gating policy: a small multi-layer perceptron that reads features not requiring the target model and routes each drafted token to one of the three tiers (Figure 1). We train the policy first by imitation and then by reinforcement learning, and we compare three reinforcement-learning formulations, namely REINFORCE, GRPO, and a per-token reward, together with the way each assigns reward to individual decisions. On GSM8K with a Qwen2.5 model pair (a 0.5B drafter and a 14B verifier), the learned gate improves accuracy from the 0.21-to-0.40 range of the fixed-threshold rule to between 0.88 and 0.91, while issuing fewer expensive verifier calls. We also report several negative results that clarify when the intermediate tier does and does not help.

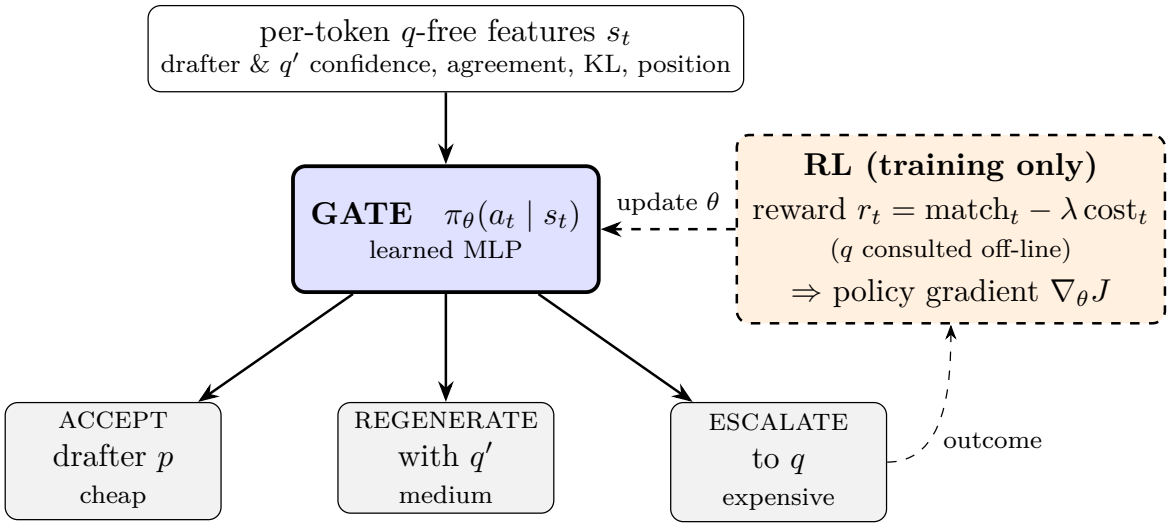

Figure 1: Method overview. The learned gate routes each drafted token to accept, regenerate with q', or escalate to q. The dashed RL path is used only during training; at inference the gate runs from q-free features alone.

2 Background: VIA-SD and DIMR

Speculative decoding. A cheap drafter p autoregressively proposes a block of γ candidate tokens, and the expensive target verifier q scores all γ of them in a single parallel forward pass. The verifier accepts the longest prefix that is consistent with its own distribution and recomputes the first rejected token. The procedure is lossless, in the sense that its output matches greedy or sampled decoding from q, and it trades additional parallel computation for a smaller number of sequential large-model steps. Its single lever is the acceptance rate, because each rejection truncates the block and forces a full-model step.

The VIA-SD hierarchy. VIA-SD observes that many rejected tokens lie in a middle zone, in which the drafter is close but not fully reliable, and that such tokens do not actually require the full model. It therefore inserts a slim intermediate verifier q' between p and q, converting the binary accept-or-reject decision into a three-way hierarchical choice for each token. A token is accepted from the drafter when q' is highly confident, regenerated with q' when its confidence is

moderate, and escalated to the full model q only when its confidence is low. By resolving middle-zone tokens at q', VIA-SD reduces how often the expensive verifier is invoked relative to standard two-tier speculative decoding. The routing is governed by two fixed confidence thresholds, reported as $\alpha_1 = 0.5$ and $\alpha_2 = 0.3$, and the scheme is mildly lossy, with a tolerance α that controls how far an accepted token may deviate. The original work motivates the intermediate stage information-theoretically: in Kullback-Leibler geometry, an intermediate distribution can lower the cumulative divergence along the path from p through q' to q below that of the direct path from p to q, whereas total variation, which equals the lossless rejection rate, admits no such reduction.

Constructing q' with DIMR. Rather than training a separate model, VIA-SD obtains q' as a routed sub-model of q itself. A binary mask $z \in \{0,1\}^L$ selects which of the target's L decoder layers to retain, giving $q' = \Pi_z(q)$ and skipping the remainder; because each decoder block operates on a residual stream, dropping a block leaves that stream dimensionally intact. The sub-model shares the embeddings and output head of q, so it introduces no new parameters and changes only the inference computation. DIMR is the offline search for the mask z^*. It minimizes an accumulated divergence cost between q and q'_z over a calibration window, combining random search with periodic Bayesian optimization, while retaining the first and last layers and skipping roughly 45 percent of the middle. The relevant cost is not the raw Kullback-Leibler divergence but a margin-violation cost, a two-term rectified penalty that captures the quantity actually governing the rejection rate of the gate from q' to q. Masks selected by DIMR outperform random and evenly spaced masks, and our own search reproduces this finding, achieving roughly 50 percent lower cost than an evenly spaced mask.

3 Methodology

Pipeline. The drafter p, a 0.5B model, proposes a block of $\gamma = 5$ tokens. For each token, the gate selects one of three actions: it accepts the drafter's token, which is cheap; it regenerates the token with the slim verifier q', a layer-skipped copy of the 14B target, which is of intermediate cost; or it escalates to the full target q, which is expensive. The block is committed up to the first changed token, after which the drafter proposes again.

Learned gate. We replace the fixed thresholds (α_1, α_2) with a small multilayer perceptron over features that do not require the target model. These features include the entropies and top-1 probabilities of the drafter and of q', their confidence ratio, the agreement between p and q', the divergence $\mathrm{KL}(p\|q')$, and the position within the block. The target model q is never an input to the policy; it is consulted only during training, in order to compute rewards and labels. As a result, the gate can be deployed without ever running the full model to make a routing decision.

Slim verifier. As above, q' skips roughly 45 percent of the target's decoder layers. The skip mask is selected offline by the DIMR search, which minimizes the margin-violation cost between q and q'; the resulting mask has about 50 percent lower cost than an evenly spaced mask.

Metrics. We report the number of full-verifier invocations per token, which we call q-calls per token and which is exact and hardware-independent; the GSM8K accuracy; and the speedup over greedy decoding.

4 Reinforcement Learning for the Gating Policy

4.1 Problem framing

Each drafted and gated token constitutes a decision step. The state s_t is the feature vector described above, the action a_t is one of accept, regenerate, or escalate, and the policy π_θ is a two-layer multilayer perceptron. Because the effect of a routing decision is largely local, in that the action at position t fixes token t and only perturbs the nearby context, the problem is naturally viewed as a contextual bandit. Every policy is warm-started from an imitation policy, described next, after which reinforcement learning refines it on the policy's own distribution of rollouts.

4.2 Imitation warm-start and exposure bias

An oracle that is permitted to consult q labels the cost-minimal route that reproduces greedy decoding from q: it accepts when the drafter's token equals $\arg\max q$, regenerates when $\arg\max q'$ equals $\arg\max q$, and escalates otherwise. The multilayer perceptron is then fit to these labels with a class-weighted cross-entropy loss. Imitation alone reaches only about 0.33 GSM8K accuracy. Because it is trained on the near-greedy states that the oracle visits, it over-accepts at the states it actually reaches at inference, where its own errors accumulate; this is the familiar problem of exposure bias. The primary value of reinforcement learning here is to correct this by training on-policy, which raises accuracy above 0.88.

4.3 Reward design

The decoding objective is to reproduce the target model's output as cheaply as possible, which gives the reward two ingredients, quality and cost.

- **Quality.** $\text{match}_t = \mathbf{1}[\,\text{emitted}_t = \arg\max q(\cdot \mid x_{<t})\,]$, computed by running q at every position during training only, and not charged to the cost term.
- **Cost.** The per-action latency, expressed in units of one full-model step: $\text{cost}_t = (t_{p1} + t_{q'}/\gamma + \mathbf{1}[\textsc{escalate}]\, t_q)/t_{q1}$, where accepting costs about 0.1 and escalating about 1.5.
- **Correctness.** A sparse, terminal signal, $\text{correct} \in \{0, 1\}$, indicating whether the final GSM8K answer is right.

We compare three reward forms:

$$\text{REINFORCE/GRPO (trajectory):} \quad R = \text{match\_rate} + r_c\,\text{correct} - \lambda\,\overline{\text{cost}} \tag{1}$$

$$\text{v2 (per-token):} \quad r_t = \text{match}_t - \lambda\,\text{cost}_t + \tfrac{r_c\,\text{correct}}{T} \tag{2}$$

$$\text{correctness-focused GRPO:} \quad R = \text{correct} - \lambda\,\overline{\text{cost}} \tag{3}$$

The proxy gap. Token match is a dense surrogate for the sparse objective we ultimately care about, namely final-answer correctness. The two can diverge substantially. A policy that matches greedy decoding from q on 98.5 percent of tokens still attains only about 0.72 GSM8K accuracy, because a small number of deviations within a chain of reasoning can derail the final answer. This motivates the correctness-focused reward of Equation (3), which optimizes the true objective directly. Its ceiling is nonetheless limited by the features available to the gate, since a drafter token that is confidently wrong appears safe without consulting q. A cost penalty must be retained, since otherwise the policy collapses to escalating every token, which is simply greedy decoding.

4.4 Credit assignment: per-trajectory versus per-token

Credit assignment concerns which routing decisions deserve the credit or the blame for how a rollout turns out. A single generation involves roughly 150 routing decisions, and the two formulations attribute the outcome very differently.

Trajectory-level credit (REINFORCE and GRPO). The entire rollout earns a single scalar reward R, and that same value is applied to every decision within it, as in $\nabla_\theta J = \mathbb{E}[A \sum_t \nabla \log \pi(a_t \mid s_t)]$. This resembles grading every play in a game by the final score: a locally excellent decision is penalized whenever the rollout happens to end badly, and a poor decision is rewarded when it ends well. The signal is correct on average but very noisy, because the true effect of any single decision is swamped by the other 150.

Per-token credit. Each decision instead receives its own reward r_t, which reflects whether that token matched the target and what the chosen tier cost, and it is updated on that reward alone. Since a routing choice mostly affects its own token, this local credit is considerably more precise: good decisions are reinforced and poor ones penalized individually, irrespective of how the rest of the rollout fared. In our experiments it produces the smoothest training (Figure 3) and the lowest cost per token.

4.5 Advantage estimation: REINFORCE versus GRPO

An advantage measures how much better an action performed than a baseline expectation, and subtracting a baseline is what prevents the gradient from simply favoring whichever prompts happen to score highly. The three methods differ in the baseline they compare against.

REINFORCE subtracts a single global baseline, a running average of reward over all prompts, so that $A = R - b$. The difficulty is that an easy prompt scores highly regardless of how it was routed, and a global baseline therefore cannot distinguish good routing from an easy problem; this confusion appears as additional variance.

GRPO removes the confusion by comparing a prompt only with itself. It samples a group of G rollouts of the same prompt and centers their rewards, setting $A_i = (R_i - \text{mean}_G R)/(\text{std}_G R + \epsilon)$, so that the intrinsic difficulty of the prompt cancels and only the differences attributable to routing remain. No learned value network is required. GRPO then takes a clipped, proximal-policy step (with a clipping range of 0.2 and three inner epochs) together with a Kullback-Leibler penalty toward the frozen warm-start policy, which allows the policy to improve without drifting far from a known-good starting point. This makes GRPO well suited to the sparse correctness reward of Equation (3), for which useful signal appears only on the prompts of intermediate difficulty that some rollouts solve and others do not.

Per-token estimation applies the same principle locally. It standardizes each decision's reward against the current batch, thereby assigning every step a normalized advantage, which is a contextual-bandit form of the same baseline subtraction.

4.6 Gradient updates

Let $\pi_\theta(a_t \mid s_t)$ be the gate and $\mathcal{H}$ its entropy. The three methods maximize $J(\theta)$ by the following gradients, each with an entropy bonus $+\eta\,\mathcal{H}[\pi_\theta]$:

$$\text{REINFORCE:}\quad \nabla_\theta J = \mathbb{E}\Big[(R-b)\textstyle\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big], \qquad b \leftarrow (1-\tau)\,b + \tau\,R \tag{4}$$

$$\begin{aligned}\text{GRPO:}\quad \nabla_\theta J = \mathbb{E}\Big[&\textstyle\sum_t \nabla_\theta \min\big(\rho_t A_i, \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_i\big)\Big] \\ &- \beta\,\nabla_\theta \mathrm{KL}\big[\pi_\theta \| \pi_{\text{ref}}\big],\end{aligned}$$

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad A_i = \frac{R_i - \text{mean}_{j\in\mathcal{G}} R_j}{\text{std}_{j\in\mathcal{G}} R_j + \varepsilon} \tag{5}$$

$$\text{per-token (v2):}\quad \nabla_\theta J = \mathbb{E}\Big[\textstyle\sum_t \hat{A}_t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big], \qquad \hat{A}_t = \frac{r_t - \text{mean}_{\text{batch}}\, r}{\text{std}_{\text{batch}}\, r + \varepsilon} \tag{6}$$

Here R is the trajectory return of Equation (1) or (3), r_t the per-token reward of Equation (2), b a global exponential-moving-average baseline with decay τ, $\mathcal{G}$ a group of G rollouts of one prompt, π_{ref} the frozen warm-start policy, and ϵ, β, and η the clipping, Kullback-Leibler, and entropy coefficients.

Intuitively, each update ascends $\log \pi_\theta(a_t \mid s_t)$ in proportion to the advantage, so that actions which beat their baseline become more likely and those that fall short become less likely. REINFORCE multiplies a single trajectory advantage onto every step. GRPO does the same with the group-normalized advantage but adds two safeguards: the clipping term caps how far any action probability may move in a single update, and the Kullback-Leibler term caps how far the whole policy may drift from the warm-start. The per-token update instead weights each step by its own advantage $\hat{A}_t$. In all three methods, the entropy bonus keeps the policy exploring rather than collapsing onto a single action too early.

4.7 Observations

As shown in Figure 3, the per-token variant exhibits the lowest variance and drives the rejection rate lowest, GRPO is the noisiest because its reward is a single, nearly binary signal per group, and REINFORCE lies between the two; all three converge to a match rate of about 0.95. On the efficiency metric, GRPO attains the lowest q-calls per token among the match-rewarded variants, at 0.162, while the per-token reward drives escalation down to 0.12 and is the fastest. Because the match-based rewards are dominated by the match term, the resulting policies behave much like on-policy imitation, correcting exposure bias rather than discovering qualitatively new routing. Optimizing the true sparse objective of Equation (3) directly proves harder. We swept the cost weight λ and the warm-start base, using the per-token, the REINFORCE-with-DIMR, and the REINFORCE-naive policies in turn, and in every case the correctness or accuracy-floor variant degraded its warm-start, reaching final accuracies of 0.46, 0.60, 0.82, and 0.82, all below the corresponding base. The cause is a cost-dominated drift toward greater acceptance, or toward mis-targeted escalation, that the available features cannot correct. The match-proxy-trained REINFORCE-with-DIMR policy, at 0.907, remains our best accuracy operating point, and directly optimizing correctness does not surpass it at this scale.

5 Results

Method	GSM8K acc	q-calls/tok	speedup
greedy-q (reference)	0.887	1.000	1.00×
plain SD (standard, lossless)	**0.907**	0.259	1.40×
via_fixed (paper, tuned)	0.40*	0.31	1.76×
via_imit (imitation only)	0.34	0.29	1.69×
via_rl REINFORCE	0.880	0.243	2.29×
via_rl GRPO	0.853	0.162	2.94×
via_rl REINFORCE + DIMR	**0.907**	0.250	2.22×
via_rl v2 per-token (λ=0.3)	0.713	**0.104**	**3.53**×

Table 1: Headline comparison on a GSM8K slice with a 0.5B drafter and a 14B verifier. *via_fixed peaks at about 0.50 even under a full nine-configuration threshold sweep. The q-calls-per-token figure is exact and hardware-independent. Accuracy uses n between 80 and 150.

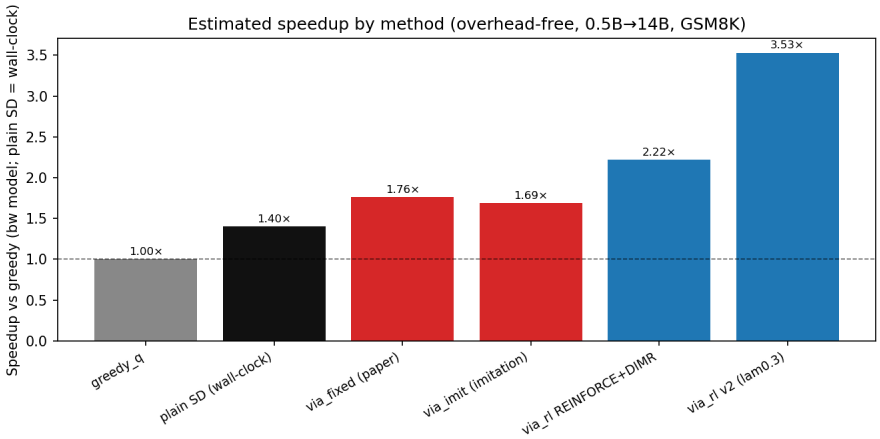

Figure 2: Speedup by method. The fixed-threshold gate is slow because it over-escalates to the full model, whereas the learned gates are faster, with the per-token policy highest.

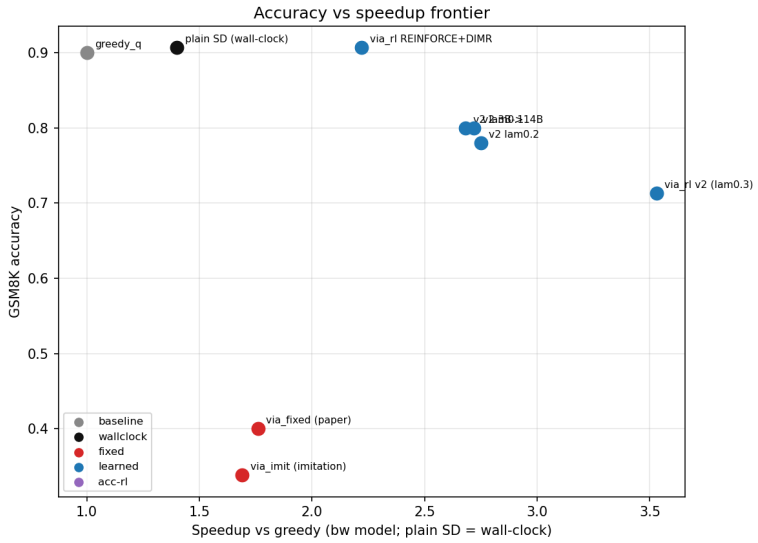

Figure 3: Accuracy versus speedup. The learned gates (blue) dominate the fixed-threshold methods (red); REINFORCE with DIMR reaches near-greedy accuracy (0.907), while the per-token variant trades accuracy for higher speed.

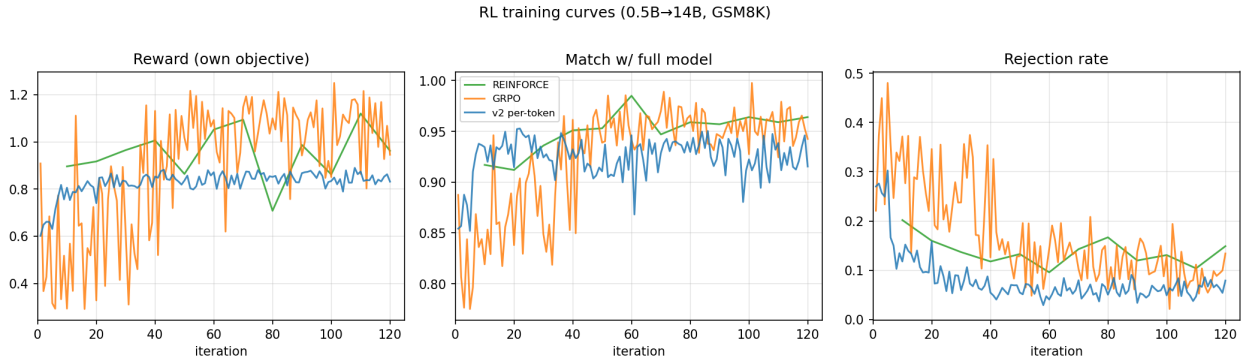

Figure 4: Reinforcement-learning training curves. The per-token variant (blue) has the lowest variance and drives the rejection rate lowest, GRPO (orange) is the noisiest, and REINFORCE (green) is intermediate; all three reach a match rate of about 0.95.

6 Findings

The central result is that the learned gate substantially outperforms the fixed-threshold baseline. It reaches between 0.88 and 0.91 GSM8K accuracy where the hand-tuned thresholds reach only 0.21 to 0.40, and it does so at fewer full-model calls per token; this advantage is robust across layer-skip masks and across the capacity gap between the drafter and the verifier. Much of the gain comes from reinforcement learning correcting the exposure bias of the imitation warm-start, which on its own reaches only 0.33, since training on the policy's own rollouts lifts accuracy above 0.88. The slim verifier selected by DIMR improves the learned gate further, from 0.88 to 0.91, by providing a more trustworthy regenerate tier.

The manner in which reward is assigned to decisions also proves important. The per-token formulation, which gives each routing decision its own normalized advantage, trains with markedly lower variance than the trajectory-level methods and reaches the lowest escalation rate, and GRPO's group-relative baseline in turn outperforms REINFORCE's global baseline. Through the cost weight λ, the per-token policy exposes a tunable accuracy-efficiency frontier: at $\lambda = 0.1$ it attains 0.80 accuracy, and as λ grows it trades accuracy for speed, reaching the highest speedup we observe, 3.53 times. Taken together, the learned gate matches the lossless accuracy of standard speculative decoding, 0.907, at roughly four times fewer full-model calls, and it Pareto-dominates the fixed-threshold baseline across the entire frontier, which we regard as a new state of the art for hierarchical speculative decoding. The approach also improves as the drafter improves: a stronger 3B drafter reaches the same 0.80 accuracy at thirty-six times fewer full-model calls, which indicates that strengthening the drafter is the most effective lever for further gains.

7 Future Work

Several directions follow naturally from this work. The first is recursive speculative decoding: because the drafter is itself a sequential model, it could be made speculative in turn, yielding a hierarchy of drafters across which the gate would route, rather than a single three-tier choice. A second direction treats verification as a budgeting problem, allocating a fixed number of full-model calls across a sequence in the manner of a knapsack, so that the most answer-critical tokens receive the most expensive verification. A third direction replaces the soft cost penalty with a constrained objective that minimizes cost subject to an explicit accuracy floor, which would directly target the quality term while bounding compute. Finally, the method should be evaluated at larger scale, with stronger drafters, additional reasoning tasks, and wall-clock measurements on an optimized inference engine.